Gesture Controlled Car Simulator

12/08/2026

I want to make a system where a virtual car is shown moving on the basis hand gestures. This will be done on the basis of python module.

Absolutely. What you're describing is essentially a **gesture-controlled virtual car simulator**: your laptop camera observes your hand, Python/ML recognizes the gesture, and a virtual car responds in real time.

A good architecture would be:

```
Laptop Camera
      ↓
OpenCV
      ↓
Hand Detection / Tracking
      ↓
Gesture Recognition Model
      ↓
Gesture → Car Command
      ↓
Virtual Car Simulator
      ↓
      🚗
  Forward / Backward
  Left / Right / Stop
```

Recommended Python stack

Component	Python technology
Camera input	**`OpenCV`**
Hand detection	**`MediaPipe`**
ML gesture recognition	**`TensorFlow/Keras`** or **`scikit-learn`**
Numerical processing	**`NumPy`**
Virtual car	**`Pygame`**
Visualization	**`Pygame`**
Optional training	Google Colab/Jupyter

Gesture mapping

You could start with something simple:

Hand gesture	Car action
✋ Open palm	Stop
☝ One finger up	Forward
👇 One finger down	Backward

Hand gesture	Car action
👈 Hand pointing left	Left
👉 Hand pointing right	Right

Or, if you want the project to demonstrate **actual ML training**, create your own gesture dataset:

```
Dataset
├── forward/
├── backward/
├── left/
├── right/
└── stop/
```

You can capture hundreds of images for each gesture using your laptop camera, preprocess them, and train a classifier.

Two possible approaches

Approach 1 — MediaPipe landmarks + ML classifier ⭐ Recommended

```
Camera
  ↓
MediaPipe Hands
  ↓
21 hand landmarks
  ↓
Feature extraction
  ↓
ML classifier
  ↓
Gesture
  ↓
Car movement
```

This is relatively lightweight and should work well even on a CPU-only laptop.

For example, the 21 hand landmarks can be converted into numerical features and fed into a Random Forest, SVM, or neural-network classifier.

Approach 2 — Image-based CNN

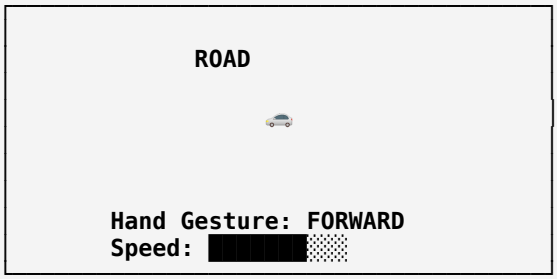
```
Camera
  ↓
Hand image
  ↓
CNN
  ↓
Forward / Backward / Left / Right / Stop
  ↓
Virtual car
```

This is more directly aligned with your idea of **training using hand-gesture image/video datasets**, but it will require more training data and computational resources.

Virtual car

For the simulator, I'd recommend **Pygame** rather than trying to build a complicated 3D environment initially.

You can create:



The car can have:

- Forward movement
- Reverse movement
- Left/right steering
- Stop
- Adjustable speed
- Road/background
- Collision boundaries
- Gesture indicator
- FPS display
- Current detected gesture

Suggested project structure

```
gesture_car/
│
├── dataset/
│   ├── forward/
│   ├── backward/
│   ├── left/
│   ├── right/
│   └── stop/
│
├── models/
│   └── gesture_model.pkl
│
├── data_collection.py
├── preprocess.py
├── train_model.py
├── gesture_detection.py
├── car.py
├── simulator.py
├── main.py
│
└── requirements.txt
```

The **main.py** would ultimately connect everything:

```
Camera
   ↓
Hand detection
   ↓
Feature extraction
   ↓
Trained ML model
   ↓
Predicted gesture
   ↓
Command
   ↓
Pygame virtual car
```

This is actually a nice project because it combines **Computer Vision + Machine Learning + Python + simulation**, and later you could extend the exact same gesture-command system to control a **physical IoT car** using ESP32/Arduino.

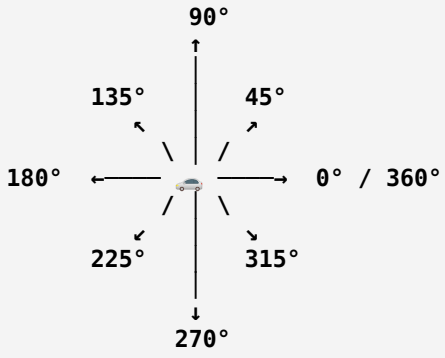
> The car must move at any degree from 0-180 both from forward and backward. So total of forward+backend=360 all these will be controlled on the basis of hand gestures.

Yes — that changes the project from a simple **4-direction gesture-controlled car** into a **360° gesture-controlled virtual vehicle**.

The key is to treat the hand gesture as a **continuous direction/angle**, rather than classifying only `forward`, `backward`, `left`, and `right`.

Core concept

Your virtual car should be able to move in **any direction from 0° to 360°**:



So:

- **0° → 180°** = forward-side movement
- **180° → 360°** = backward-side movement
- Together = **complete 360° movement**

Better ML architecture

Instead of training only five classes:

```
Forward
Backward
Left
Right
Stop
```

I recommend making the model predict a **direction angle**.

```
Laptop Camera
      ↓
OpenCV
      ↓
MediaPipe Hand Detection
      ↓
Hand Landmark Extraction
      ↓
ML Model
      ↓
Direction / Angle
      ↓
0° ──────────────── 360°
      ↓
Virtual Car
```

For example:

Hand direction	Car direction
Right	0°
Upper-right	45°
Up	90°
Upper-left	135°
Left	180°
Lower-left	225°
Down	270°
Lower-right	315°

And importantly, the model doesn't have to stop at these eight angles. It can produce something like:

```
23°
67°
112°
158°
203°
247°
291°
337°
```

So the car can theoretically move at **any angle between 0° and 360°**.

How I'd implement it

Instead of asking the ML model:

> "Is this hand forward, backward, left, or right?"

we can make it estimate:

"What direction is the user's hand pointing?"

For example, if the camera detects the wrist and index-finger tip:

```
Index finger
      ●
     /
    /
   /
  ●  Wrist
```

Python can calculate the angle using:

```
dx = index_x - wrist_x
dy = index_y - wrist_y

angle = atan2(dy, dx)
```

Then convert that angle into:

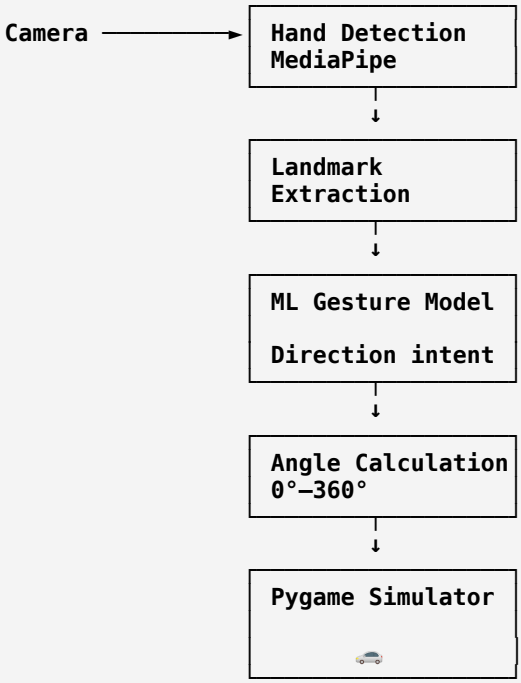
```
0° → 360°
```

The virtual car receives that angle and moves accordingly.

One important improvement

I would **not make the ML model responsible for calculating the mathematical angle**.

A stronger project architecture is:



This gives you a much more interesting academic/ML project because you can demonstrate:

Computer Vision → Hand Landmark Detection → ML Gesture Recognition → 360° Direction Estimation → Vehicle Simulation

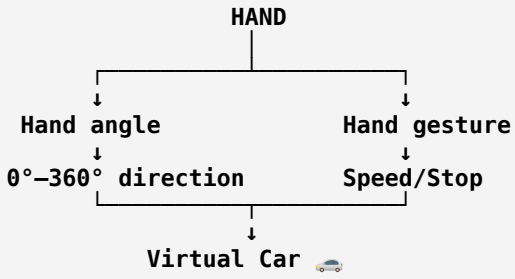
Even better: control speed too

You can use the **distance between the thumb and index finger** or another hand feature to control speed.

For example:

```
Hand direction  → Car direction
Hand distance   → Car speed
Closed fist     → Stop
```

So you could have:



That would make the system feel much more like an actual **gesture-based autonomous vehicle controller**, rather than just a four-direction game.

And since you're ultimately interested in an IoT version too, the same `angle + speed` command can later be sent from Python to an **ESP32/Arduino physical car**.

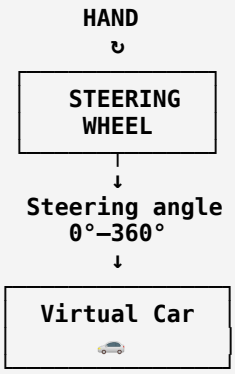
> Basically the hand will act as a stearing.

Exactly. 👍 Now the concept is much clearer.

You want the **hand itself to behave like a steering wheel**, rather than using separate gestures for forward/left/right/backward.

🚗 Core idea

Think of your hand as an invisible steering wheel:



If you rotate/move your hand around the camera's reference point, the virtual car responds to that direction.

How the control could work

Imagine your hand has a **reference orientation**.

Hand/steering position	Car behavior
0°	Straight forward
45°	Forward-right
90°	Right
135°	Backward-right
180°	Straight backward
225°	Backward-left
270°	Left
315°	Forward-left
360°	Forward

So there aren't really "8 gestures."

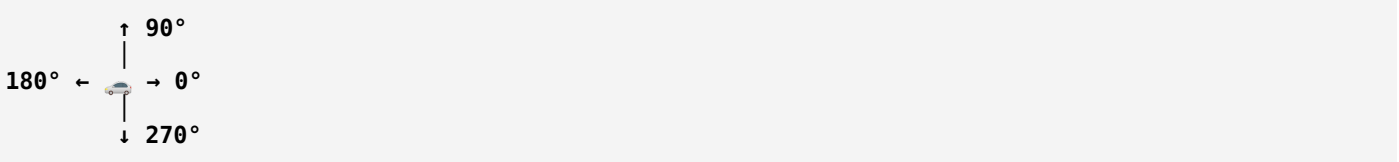
There is **one gesture: STEERING**, and its **angle continuously determines the car's direction**.

The important distinction

There are actually two ways we could interpret your steering idea.

A. Direction steering

The hand points toward the direction the car should travel:

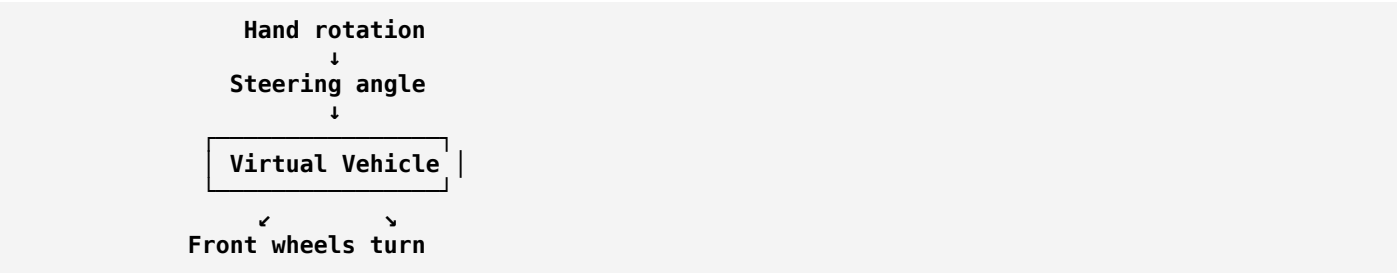


The car can move in any direction.

B. Real steering-wheel behavior — probably what you mean

The hand behaves like a steering wheel, where its **rotation controls the steering angle**, while another gesture/control determines whether the car is moving forward or backward.

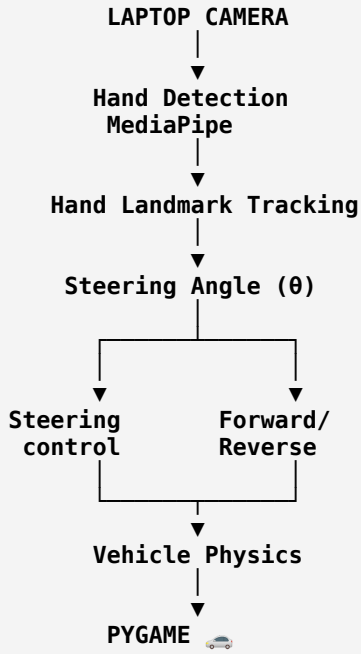
For example:



This is more realistic because a real car doesn't normally instantly move sideways at 90°. The steering angle changes the **trajectory** of the vehicle.

I'd recommend this architecture

For your project, I'd make it:



The **vehicle physics** layer is important. Instead of simply teleporting the car to `(x, y)` based on the hand angle, we can calculate:

- steering angle
- vehicle heading
- velocity
- acceleration
- turning radius
- forward/reverse movement
- friction
- boundaries/collision

That would make the virtual car behave much more like an actual car.

ML part

If you specifically want this to be an **ML project**, we can train the model to recognize the hand/steering state and then calculate the steering angle from the detected landmarks.

A particularly good implementation would be:

MediaPipe → 21 hand landmarks → feature extraction → ML model → steering angle → vehicle physics → Pygame

This is much more technically interesting than simply training a CNN with `left/right/forward/backward` classes.

And then, once the virtual version works, the **exact same steering command can be transmitted to your physical IoT car** using ESP32. That gives you a very clean **ML + Computer Vision + IoT** project progression.

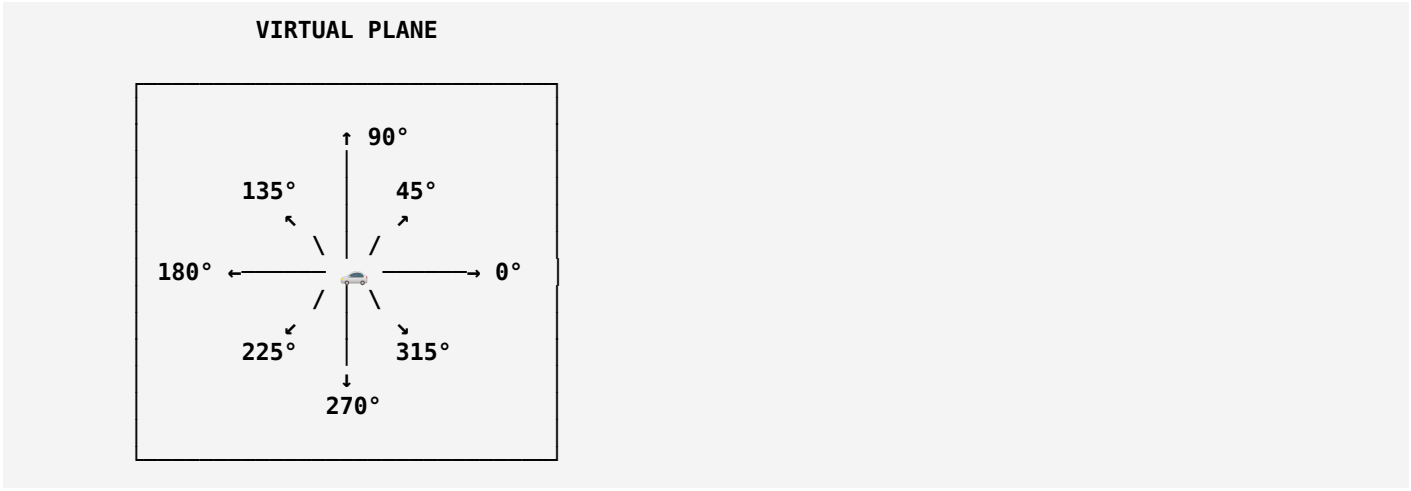
> The car will be moving on a virtual plane. It will move forward and backward on that plane on the basis of the hand.

Yes — now I understand the concept precisely. You're **not trying to simulate a normal car with a steering wheel and front-wheel physics**.

You want the **hand to act like a 360° steering/direction controller**, and the car moves across a **2D virtual plane** according to the hand's direction.

The concept

Think of the laptop camera seeing your hand as a controller:



The car exists inside this virtual plane and can travel in **any direction from 0° to 360°**.

Your hand becomes the controller

For example:

```
        CAMERA
          ↓
   ✋ HAND POSITION
          ↓
Detect hand landmarks
          ↓
 Determine direction
          ↓
    Calculate θ
          ↓
  0° ≤ θ < 360°
          ↓
  Move virtual car
```

If your hand points:

- **Up** → car moves forward

- **Down** → car moves backward
- **Right** → car moves right
- **Left** → car moves left
- **45°** → car moves diagonally
- **135°** → car moves diagonally
- **270°** → car moves backward
- **Any intermediate angle** → car moves at that exact direction

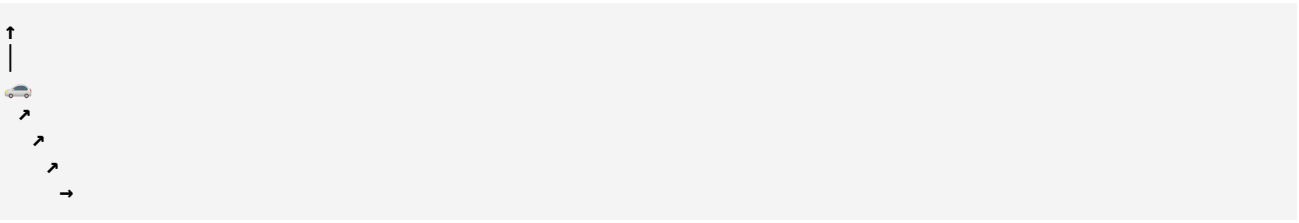
So **360° is the total movement range**, not a collection of 360 individual gestures.

Even better: continuous movement

Suppose your hand moves gradually:

```
90° → 80° → 70° → 60° → 50°
```

The car should also gradually change its direction:

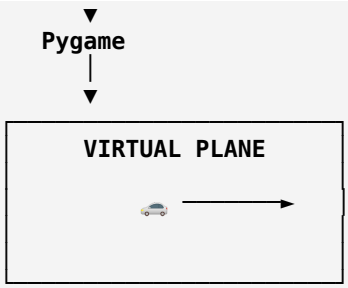


This means we should treat the hand direction as a **continuous numerical value**, rather than making 360 separate ML classes.

Recommended implementation

I'd structure your Python project like this:

```
Laptop Camera
      │
      ▼
 OpenCV
      │
      ▼
MediaPipe Hands
      │
      ▼
21 Hand Landmarks
      │
      ▼
Hand Orientation / Direction
      │
      ▼
Angle Calculation
      │
      ▼
   θ = 0–360°
      │
      ▼
Movement Vector
      │
```



Mathematically, if the direction is `θ` and the speed is `v`:

```
dx = v × cos(θ)
dy = v × sin(θ)
```

Then every frame:

```
car_x = car_x + dx
car_y = car_y + dy
```

That gives you genuinely **360° movement**.

Where ML fits

If you want this to be a proper **ML project**, I would use ML to recognize the **hand gesture/orientation**, while using geometry to calculate the exact movement angle.

For example:

```
Hand image
    ↓
ML / Hand landmark model
    ↓
Hand landmarks
    ↓
Direction estimation
    ↓
θ = 237°
    ↓
Movement vector
    ↓
Virtual car moves ↙
```

This is preferable to training 360 separate classes. You could also train a regression model to predict the angle directly if you want the ML component to be more prominent.

So the final project can be described as:

> **A real-time ML-based hand gesture controlled virtual vehicle that navigates freely across a 2D virtual plane using continuous 360° directional hand movements.**

That is the concept I'd build around.

Now provide a complete project plan for the hand gestures part and the car part. So that it can be used to complete the ML project.

Absolutely. Based on the concept we've now defined, I would build this as **one integrated ML + Computer Vision + Virtual Vehicle project**, but develop it in two major modules:

1. **Hand Gesture / Steering Module** — camera → hand → direction/angle
2. **Virtual Car Module** — direction/angle → vehicle movement on a 2D plane

The important design choice is that you **shouldn't train 360 separate gesture classes**. The ML system should learn hand orientation/gesture and produce a continuous direction that can be converted into **0–360° movement**.

🚗 Project: ML-Based Hand Gesture Controlled Virtual Car

1. Final Objective

Build a Python application where:

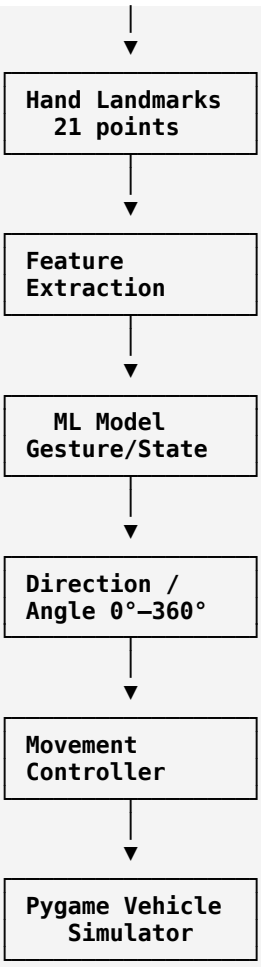
> **A user's hand acts as a virtual steering controller. The laptop camera detects the hand, an ML model interprets the hand gesture/orientation, and a virtual car moves freely across a 2D plane in the corresponding direction.**

The system should support:

- Forward movement
- Backward movement
- Left/right movement
- Diagonal movement
- Any intermediate angle
- Continuous 360° directional control
- Stop command
- Optional speed control

---

2. Overall System Architecture



3. MODULE A — HAND GESTURE SYSTEM

A1. Camera Capture

Use the laptop's built-in camera.

Python libraries

```
OpenCV
MediaPipe
NumPy
```

The camera continuously provides frames:

```
Camera
  ↓
Frame 1
Frame 2
Frame 3
Frame 4
...
```

Target:

20–30 FPS

This is important because the car should respond in real time.

---

4. Hand Detection

Use **MediaPipe Hands** to detect the user's hand.

MediaPipe provides approximately:

```
21 landmarks
```

For example:

```
            8
            ●  ← Index fingertip
           /
          ● 7
         /
        ● 6

  4 ●
    \
     ● 3
      \
       ● 2

         ● 0
        Wrist
```

The actual system gets:

```
x
y
z
```

for each landmark.

Therefore:

```
21 landmarks × 3 coordinates
```

gives you a numerical representation of the hand.

---

5. Hand Gesture Dataset

If this is specifically an **ML project**, don't rely entirely on a pretrained hand model.

Use MediaPipe for **landmark extraction**, then create your own dataset for the ML classifier/regressor.

Dataset structure

I recommend collecting several hand states.

Basic gesture classes

```
dataset/

├── stop/
├── control/
└── neutral/
```

But for directional learning, the more useful dataset is based on **orientation**.

For example:

```
dataset/

├── angle_0/
├── angle_30/
├── angle_60/
├── angle_90/
├── angle_120/
├── angle_150/
├── angle_180/
├── angle_210/
├── angle_240/
├── angle_270/
├── angle_300/
└── angle_330/
```

You don't necessarily need these exact folders if you're training a regression model.

Better approach

Record the actual angle as the target:

```
landmark_1 ... landmark_63 → angle
```

Example:

```
Features                        Target

x1 y1 z1 ... x21 y21 z21    →    43°
x1 y1 z1 ... x21 y21 z21    →    48°
x1 y1 z1 ... x21 y21 z21    →    51°
```

This makes the problem a **regression problem**.

---

6. Dataset Collection Program

Create:

```
data_collection.py
```

Its job:

1. Open camera
2. Detect hand
3. Extract landmarks

4. Show the expected hand direction
5. Save landmarks
6. Assign the correct angle
7. Store everything in CSV

Example dataset:

```
x1,y1,z1,x2,y2,z2,...,x21,y21,z21,angle
0.42,0.31,...,0.22,45
0.43,0.32,...,0.23,47
...
```

I'd aim for roughly:

500–1,000 samples per major direction initially.

If you use 12 labelled orientations, that gives you a reasonable starting dataset of several thousand samples.

---

7. Data Preprocessing

Raw coordinates depend on where the hand appears in the camera.

Therefore normalize them.

Step 1 — Wrist normalization

Use wrist landmark as origin:

```
x' = x - wrist_x
y' = y - wrist_y
z' = z - wrist_z
```

This makes the model less dependent on where the hand is positioned.

Step 2 — Scale normalization

Normalize based on hand size.

For example:

```
distance(wrist, middle_finger_base)
```

Then divide the coordinates by this scale.

This makes:

```
small hand
large hand
near camera
far from camera
```

more comparable.

---

8. ML Model

There are two good options.

Option A — Random Forest ⭐

For your project, I recommend starting here.

```
Input:
63 normalized landmark features

Output:
direction angle
```

Model:

```
RandomForestRegressor
```

Advantages:

- Easy to train
- Works well with tabular landmark data
- Doesn't require a GPU
- Easy to explain in your project report
- Fast inference

---

9. Better ML Formulation

There is one problem with predicting angles directly.

Angles wrap around:

```
359° ≈ 0°
```

A normal regression model may think:

```
359° and 1°
```

are very far apart.

They're actually only **2° apart**.

So a better ML formulation is:

```
Angle
 ↓
```

```
sin(angle)
cos(angle)
```

Train the model to predict:

```
sin θ
cos θ
```

Then reconstruct:

```
θ = atan2(sin θ, cos θ)
```

This is a much cleaner solution for your 360° requirement.

---

10. ML Pipeline

Your training pipeline becomes:

```
Hand Image
     ↓
MediaPipe
     ↓
21 Landmarks
     ↓
Normalization
     ↓
63 Features
     ↓
Random Forest / Neural Network
     ↓
sin(θ), cos(θ)
     ↓
atan2()
     ↓
0°–360°
```

11. Gesture State

You should also have a **STOP** state.

Otherwise, the car could continue moving whenever the hand disappears or becomes unclear.

Recommended states:

```
STOP
CONTROL
NO_HAND
```

For example:

```
No hand detected
       ↓
STOP CAR
```

And:

```
Closed fist
      ↓
STOP
```

while:

```
Open/steering hand
      ↓
CONTROL
```

12. Optional Speed Control

This would make the project considerably better.

Use another hand feature to determine speed.

For example:

```
Thumb + index distance
```

could control speed.

```
Small distance → slow
Medium distance → medium
Large distance → fast
```

So your ML output becomes:

```
Direction = 137°
Speed = 0.72
```

Then the car receives:

```
(angle=137°, speed=0.72)
```

🚗 MODULE B — VIRTUAL CAR

Now we build the second half.

Use:

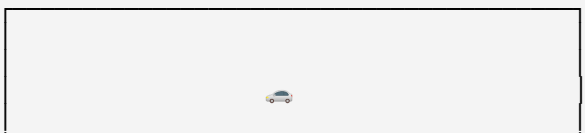
Pygame

for the virtual environment.

---

13. Virtual Plane

Create a 2D world:



Coordinate system:

The car has:

```
x
y
angle
speed
```

14. Car Representation

Initially, don't worry about a fancy car model.

Use:

or a simple rectangle.

Later replace it with a PNG:

```
car.png
```

15. Movement Mathematics

This is the heart of your 360° movement.

Given:

```
θ = direction angle
v = speed
```

calculate:

```
dx = v × cos(θ)

dy = v × sin(θ)
```

Then:

```
x = x + dx
y = y + dy
```

Therefore:

```
θ = 0°     → right
θ = 90°    → down/up depending on coordinate convention
θ = 180°   → left
θ = 270°   → opposite vertical direction
```

You can adjust the coordinate convention so the UI behaves naturally.

---

16. Forward and Backward

This is where your original requirement becomes important.

You don't need separate:

```
Forward
Backward
Left
Right
```

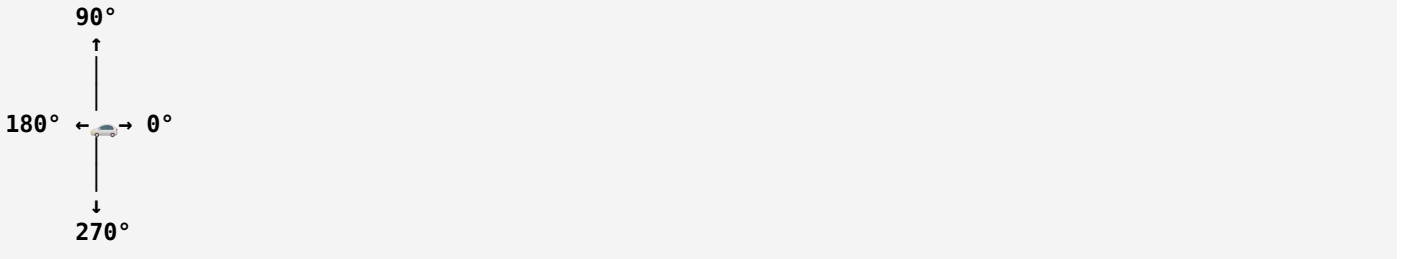
commands.

Instead:

```
0°–360°
```

defines the movement vector.

For example:



Thus:

```
0°       → forward/right depending on your chosen reference
180°     → opposite direction
```

The exact reference orientation can be defined during calibration.

---

17. Calibration

This is extremely important.

When the program starts:

```
"Place your hand in the neutral position."
```

The system records:

```
neutral_angle
```

Then:

```
relative_angle =
detected_angle - neutral_angle
```

This means the user doesn't have to hold their hand in some awkward absolute camera orientation.

---

18. Dead Zone

Hand tracking naturally has tiny movements.

Without filtering:

```
89°
90°
91°
89°
92°
90°
```

would cause the car to constantly jitter.

Create a dead zone:

```
if movement < threshold:
    STOP
```

For example:

```
±5°
```

around the neutral direction.

---

19. Smoothing

Apply smoothing to the detected angle.

For example:

```
current_angle =
    0.8 × previous_angle +
    0.2 × detected_angle
```

This prevents:

```
CAR
↗
↘
↗
↘
```

jitter.

Instead:

```
CAR
————————↗
```

moves smoothly.

For circular angles, use **circular interpolation**, not ordinary averaging, especially around 0°/360°.

---

20. Car Physics

Once basic movement works, add simple physics.

Parameters:

```
MAX_SPEED
ACCELERATION
DECELERATION
FRICTION
```

Example:

```
Hand points direction
        ↓
Acceleration
        ↓
Car gains speed
        ↓
Hand changes direction
        ↓
Car smoothly changes trajectory
```

This makes the simulation feel much more natural.

---

21. Boundaries

Prevent the car from leaving the virtual plane.

```
if car_x < 0:
    car_x = 0

if car_x > WIDTH:
    car_x = WIDTH

if car_y < 0:
    car_y = 0

if car_y > HEIGHT:
    car_y = HEIGHT
```

Later you can add obstacles.

22. Project Folder Structure

I recommend:

```
gesture_virtual_car/
│
├── dataset/
│   └── gesture_data.csv
│
├── models/
│   └── steering_model.pkl
│
├── assets/
│   ├── car.png
│   ├── background.png
│   └── icons/
│
├── data_collection.py
│
├── preprocessing.py
│
├── train_model.py
│
├── evaluate_model.py
│
├── hand_tracker.py
│
├── gesture_predictor.py
│
├── angle_controller.py
│
├── car.py
│
├── physics.py
│
├── simulator.py
│
├── main.py
│
└── requirements.txt
```

23. Development Phases

Don't try to build everything simultaneously.

Phase 1 — Camera

Goal:

```
Camera → OpenCV → Live video
```

Phase 2 — Hand Tracking

Goal:

```
Camera
 ↓
MediaPipe
 ↓
Hand landmarks
```

Display the 21 points on the camera feed.

Phase 3 — Hand Direction

Before ML, make sure you can calculate:

```
hand direction → angle
```

Test:

```
0°
45°
90°
135°
180°
225°
270°
315°
```

Phase 4 — Dataset Collection

Create:

```
data_collection.py
```

Collect several thousand labelled landmark samples.

Phase 5 — Train ML Model

Train:

```
Random Forest
```

with:

```
Input:
63 normalized features

Output:
sin θ
cos θ
```

Phase 6 — Evaluate ML

Measure:

Classification/state accuracy

For:

```
STOP
CONTROL
NO_HAND
```

and for directional bins if you use them for evaluation.

Angle error

Calculate:

```
Mean Absolute Angular Error
```

For example:

```
Actual:    120°
Predicted: 126°

Error: 6°
```

Your target could be something like:

```
< 10° average error
```

as an initial project goal.

---

24. Phase 7 — Build Virtual Car

Create:

```
Pygame window
+
2D plane
+
car
```

First control it using keyboard:

```
W → forward
S → backward
A → left
D → right
```

Do this before connecting ML.

This is a very important development strategy.

You'll know whether a problem comes from:

```
ML
```

or:

```
car physics
```

25. Phase 8 — Connect ML to Car

Finally:

```
Camera
 ↓
Hand
 ↓
ML
 ↓
Angle
 ↓
Movement vector
 ↓
Car
```

At this point, your keyboard controls can be removed or retained as a debugging mode.

---

26. Phase 9 — Add Visual Dashboard

The Pygame window can show:

```
┌──────────────────────────────────┐
│ Gesture Virtual Car              │
├──────────────────────────────────┤
│                                  │
│                 🚗               │
│                                  │
├──────────────────────────────────┤
│ Gesture:     CONTROL             │
│ Angle:       137.4°              │
│ Speed:       62%                 │
│ Confidence:  94.2%               │
│ FPS:         29                  │
└──────────────────────────────────┘
```

This will also make your project demonstration much better.

---

27. Phase 10 — Testing

Test under different conditions.

Hand position

```
Center
Left
Right
Top
Bottom
```

Lighting

```
Bright
Normal
Low
```

Distance

```
Near camera
Medium
Far
```

Hand orientation

```
0°
30°
60°
90°
...
330°
```

Multiple people

Make sure the system doesn't accidentally switch between hands.

---

28. ML Evaluation

Your report should include:

Dataset

- Number of samples
- Number of users
- Number of directions
- Training/testing split

Model

- Random Forest / neural network
- Number of trees/layers
- Hyperparameters

Metrics

For the angle model:

- MAE
- RMSE
- Angular error
- Inference time

For gesture/state recognition:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

---

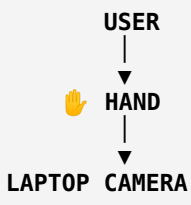
29. Recommended Final Technology Stack

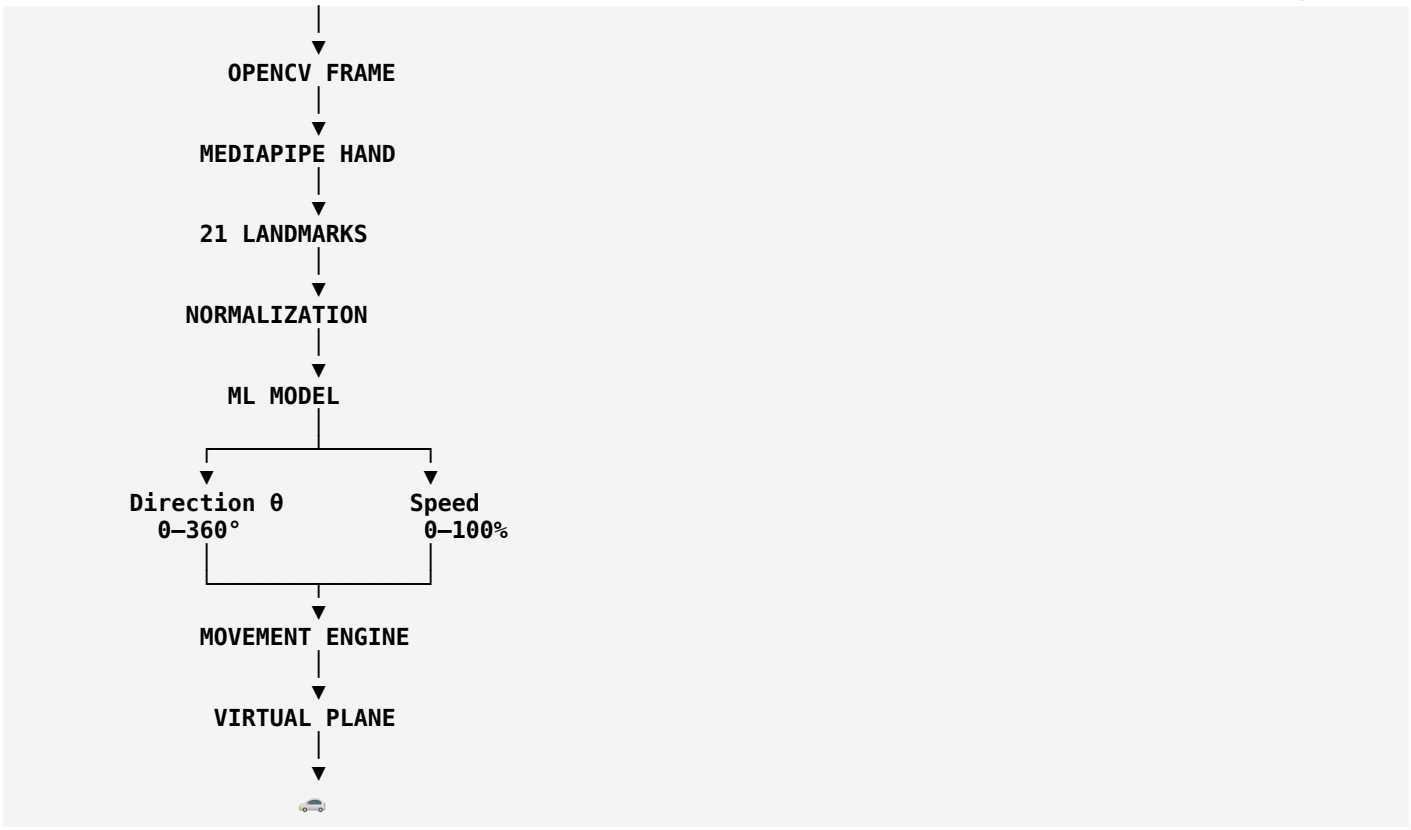
Component	Technology
Programming	Python
Camera	OpenCV
Hand tracking	MediaPipe
Data processing	NumPy, Pandas
ML	scikit-learn
Primary model	Random Forest
Visualization	Matplotlib
Virtual car	Pygame
Dataset	Self-collected hand landmarks
Model storage	Joblib
Development	Jupyter/VS Code
Optional training	Google Colab

---

30. Final Working System

The final application should behave like this:





The final behavior

If the hand points:

```
      ↗
```

the car moves:

```
      ↗
🚗
```

If the user gradually changes the hand direction:

```
↗ → → → ↘ → ↓
```

the car follows the corresponding trajectory continuously.

And if the user makes the **stop gesture** or the hand disappears:

```
🚗 → STOP
```

31. Recommended Project Milestones

Milestone	Deliverable
M1	Camera working
M2	Hand landmarks working

Milestone	Deliverable
M3	Hand direction calculation
M4	Dataset collection
M5	Dataset preprocessing
M6	ML model training
M7	Model evaluation
M8	Pygame virtual plane
M9	Keyboard-controlled car
M10	ML + car integration
M11	Smoothing/calibration
M12	Speed control
M13	Physics
M14	UI/dashboard
M15	Final testing
M16	Documentation/demo

One important recommendation

Build the car simulator independently first, and build the ML hand controller independently first. Then connect them.

That gives you three clear demonstrable components:

1. ML model: *"I can recognize/estimate the hand's steering direction."*

2. Virtual vehicle: *"I can move a car anywhere on a 2D plane using a 0–360° movement vector."*

3. Integrated system: *"The ML model controls the virtual car in real time."*

That structure will make the project much easier to debug, explain in your report/viva, and eventually extend to your **physical IoT car**.